Few-View Computed Tomography — Introduction

Reproducibility material for the large-scale matrix-free CT experiment

1. Purpose of this folder

This folder contains the computational material for the few-view computed tomography (CT) experiment described in Section 4.3 of the dissertation. The purpose of the material is to make the experiment understandable and reproducible without requiring the reader to reconstruct the implementation from the dissertation alone.

The benchmark compares four iterative methods — CG, PCG, LSQR and LSMR — on the same large, matrix-free CT reconstruction problem. The supplied Python program creates the synthetic CT problem, applies the four solvers for the specified regularisation values, records the measurements, and writes CSV files containing the results. A separate plotting program reads those CSV files and creates the figures used to inspect the solver behaviour.

The material is written so that a reader who is not a programmer can follow the purpose of each file and understand what is being reproduced. A working knowledge of basic command-line use is sufficient; the instructions below explain the commands that are required.

2. What the CT problem represents

Computed tomography reconstructs an image from measurements obtained by projecting the object from different directions. In this experiment the unknown object is a two-dimensional 1024 by 1024 image. Each image pixel is an unknown quantity, so the reconstruction contains 1,048,576 unknowns.

Only 64 projection views are used. Each view produces 1024 detector measurements, giving 65,536 measurements in total. Thus the problem has far fewer measurements than unknowns: m/n = 65,536 / 1,048,576 = 0.0625. This is a severely underdetermined inverse problem. The experiment is therefore useful for examining how the four iterative methods behave when the available CT information is limited.

The CT operator is parallel-beam and matrix-free. Rather than constructing and storing a conventional CT system matrix A, the code represents A through operations that apply the forward projection and its adjoint. This is important at this scale because an explicit matrix would have an extremely large number of entries and would defeat the purpose of the large-scale matrix-free experiment.

3. The reconstruction problem

The reconstruction uses Tikhonov-type L2 regularisation. The objective is

```
minimise over x: ||A x - b||^2 + lambda ||x||^2
```

Here A represents the matrix-free CT forward operator, x is the unknown image, and b is the measured projection data. The first term measures how well a reconstructed image explains the CT measurements. The second term penalises the size of the reconstructed solution. The parameter lambda controls the strength of this regularisation.

The supplied implementation solves the regularised problem in two related ways. CG and PCG operate on the regularised normal-equation operator A transpose A + lambda I. LSQR and LSMR instead operate on an augmented least-squares operator containing A together with the square-root regularisation term. This keeps the least-squares methods in their intended matrix-free formulation rather than explicitly forming the normal-equation matrix.

4. Synthetic image and measurements

The Python program constructs a Shepp-Logan-type synthetic phantom as the reference image. The phantom provides a known ground-truth image, which makes it possible to calculate a relative reconstruction error after solving the inverse problem.

The clean projection data are obtained by applying the matrix-free CT forward operator to the reference image. Controlled Gaussian noise is then added. The supplied benchmark uses a noise level of 0.002, with the generated noise scaled relative to the norm of the clean projection data. A fixed random seed is supplied so that the noise generation is controlled rather than left entirely to chance.

For every regularisation value, the benchmark performs two repeats. The four solvers therefore produce repeated measurements of runtime, iteration count and reconstruction quality. The summary CSV averages the repeated runs.

5. Benchmark settings

Quantity	Value used by supplied benchmark
Image	1024 x 1024
Unknowns	1,048,576
Views	64
Measurements	65,536
m/n	0.0625
Geometry	Parallel-beam CT
Operator	Matrix-free
Solvers	CG, PCG, LSQR, LSMR
Regularisation	1e-6, 1e-5, 1e-4, 1e-3, 1e-2
Noise level	0.002
Repeats	2
Tolerance	1e-6
Maximum iterations	120
Reference image	Shepp-Logan-type phantom

These values are part of the benchmark definition. Changing them produces a different experiment. In particular, changing the image size, number of views, noise level, regularisation sweep, tolerance or iteration limit should not be described as an exact reproduction of the dissertation experiment.

6. What each file does

File	Purpose
CT.py	Creates the synthetic CT problem, runs CG, PCG, LSQR and LSMR, and writes the numerical results.
plot_CT.py	Reads the generated CSV files and produces the plots and a short plotting summary.
CT_Introduction.pdf	Explains the experiment and its role in the dissertation.
CT_Run_Instructions.pdf	Gives step-by-step instructions for running the benchmark and plotting programs.
CT_Requirements_and_Restrictions.pdf	Records software, hardware, memory, CPU, matrix-free and reproducibility requirements.

7. What the benchmark records

The main numerical output is CT_solver_results.csv. It contains individual solver results for each regularisation value and repeat. CT_solver_summary.csv groups those results by regularisation value and solver and calculates mean and standard-deviation quantities. CT_solver_runtime_comparison.csv provides runtime and iteration ratios involving LSQR.

The plotting program uses these CSV files rather than rerunning the CT solver. It creates ten PNG figures, including runtime versus regularisation, reconstruction error, data-fit residual, normal-equation residual, best reconstruction error, fastest observed runtime, LSQR runtime ratios, iterations versus error, runtime versus accuracy, and a grouped runtime comparison.

8. Interpreting the dissertation result

The dissertation reports that CG, LSQR and LSMR produced closely grouped reconstruction errors, while PCG was substantially less accurate for the reported experiment. The reported best reconstruction errors for CG and LSQR occur at lambda = 1e-5, with values of 0.239554 and 0.239556 respectively. The dissertation therefore uses this experiment to show that LSQR can achieve essentially the same reconstruction accuracy as CG while retaining a matrix-free least-squares formulation.

The dissertation also reports that the four methods reached the imposed limit of 120 iterations with a convergence rate of zero. This means that, in the recorded runs, the iteration cap was reached rather than the requested stopping tolerance being satisfied. Runtime measurements should be treated as hardware- and environment-dependent quantities; reproducing the computational procedure does not guarantee identical wall-clock times.

9. Reproducibility principle

The most important point when using this folder is to distinguish reproducing the experiment from modifying it. The supplied code should be treated as the reference implementation. A researcher may change settings for a new investigation, but those runs should be labelled as modified experiments rather than presented as direct reproductions of the dissertation benchmark.

The generated CSV and PNG files do not need to remain in the repository if storage is a concern. They can be regenerated from the supplied Python files. The essential reproducibility material is the code, the documentation of the benchmark parameters, and the information needed to recreate the computational environment.